

AFRILANG Stdlib

std/c/encryptcaesar

Bibliothèque AFRILANG `std/c/encryptcaesar` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : encrypt, decrypt

```
`import "std/c/encryptcaesar" . `use encryptcaesar`
```

- `encrypt(s text, shift number) ? text`
- `decrypt(s text, shift number) ? text`
- `bruteForce(s text) ? list text`
- `countAlpha(s text) ? number`
- `rotAll(s text) ? list text`
- `isCaesar(plain text, cipher text) ? bool`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/encryptcaesar` (niveau complexe, catégorie complexe). Elle expose 6 fonction(s) : encrypt, decrypt

```
`import "std/c/encryptcaesar"` . `use encryptcaesar`  
- `encrypt(s text, shift number) ? text`  
- `decrypt(s text, shift number) ? text`  
- `bruteForce(s text) ? list text`  
- `countAlpha(s text) ? number`  
- `rotAll(s text) ? list text`  
- `isCaesar(plain text, cipher text) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/encryptcaesar"  
use encryptcaesar
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? encrypt(s text, shift number) ? text  
? decrypt(s text, shift number) ? text  
? bruteForce(s text) ? list  
? countAlpha(s text) ? number  
? rotAll(s text) ? list  
? isCaesar(plain text, cipher text) ? bool
```

4. Exemples d'utilisation

```
import "std/c/encryptcaesar"  
use encryptcaesar  
  
create result = encrypt(/* args */)   
say result  
  
create result = decrypt(/* args */)   
say result  
  
create result = bruteForce(/* args */)   
say result  
  
create result = countAlpha(/* args */)   
say result  
  
create result = rotAll(/* args */)   
say result  
  
create result = isCaesar(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/encryptcaesar"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module encryptcaesar  
  
function encrypt(s text, shift number) returns text  
end
```

```
function decrypt(s text, shift number) returns text
end

function bruteForce(s text) returns list text
end

function countAlpha(s text) returns number
end

function rotAll(s text) returns list text
end

function isCaesar(plain text, cipher text) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/encryptcaesar` (tier complex, category complex). Exports 6 function(s): encrypt, decrypt, bruteFor

```
`import "std/c/encryptcaesar"` . `use encryptcaesar`  
- `encrypt(s text, shift number) ? text`  
- `decrypt(s text, shift number) ? text`  
- `bruteForce(s text) ? list text`  
- `countAlpha(s text) ? number`  
- `rotAll(s text) ? list text`  
- `isCaesar(plain text, cipher text) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/encryptcaesar"  
use encryptcaesar
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? encrypt(s text, shift number) ? text  
? decrypt(s text, shift number) ? text  
? bruteForce(s text) ? list  
? countAlpha(s text) ? number  
? rotAll(s text) ? list  
? isCaesar(plain text, cipher text) ? bool
```

4. Usage examples

```
import "std/c/encryptcaesar"  
use encryptcaesar  
  
create result = encrypt(/* args */)   
say result  
  
create result = decrypt(/* args */)   
say result  
  
create result = bruteForce(/* args */)   
say result  
  
create result = countAlpha(/* args */)   
say result  
  
create result = rotAll(/* args */)   
say result  
  
create result = isCaesar(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/encryptcaesar"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module encryptcaesar  
  
function encrypt(s text, shift number) returns text  
end
```

```
function decrypt(s text, shift number) returns text
end

function bruteForce(s text) returns list text
end

function countAlpha(s text) returns number
end

function rotAll(s text) returns list text
end

function isCaesar(plain text, cipher text) returns bool
end
end
```